

Guía Práctica de Software Testing

Las pruebas de software efectivas se basan en un enfoque equilibrado a través de tres niveles distintos. Cada nivel cumple un propósito único para garantizar la fiabilidad y calidad de la aplicación.

Pruebas Unitarias Prueban funcionalidades aisladas y pequeñas unidades de código. Validan la lógica sin dependencias externas como bases de datos o APIs.	Pruebas de Integración Verifican cómo múltiples módulos o componentes interactúan entre sí, incluyendo dependencias externas y flujos de datos.	Pruebas End-to-End (E2E) Prueban la aplicación completa desde la perspectiva del usuario, simulando escenarios de uso y flujos de trabajo del mundo real.
---	---	---

La Distribución Ideal de las Pruebas



Esta distribución maximiza la cobertura de pruebas mientras minimiza el tiempo de ejecución y la sobrecarga de mantenimiento. Las pruebas unitarias proporcionan retroalimentación rápida, mientras que un número menor de pruebas E2E validan rutas críticas del usuario.

Herramientas Esenciales de Prueba por Categoría

Pruebas Unitarias Jest – Framework todo en uno, perfecto para REST APIs con mocking y aserciones integrados. Vitest – Alternativa ultrarrápida con una API similar a Jest, excelente para proyectos JavaScript modernos. Mocha + Chai – Combinación altamente flexible ideal para proyectos a gran escala que requieren configuraciones personalizadas.	Pruebas de Integración Jest / Vitest – Extiende frameworks de pruebas unitarias para escenarios de integración. MongoDB Memory Server – Simula una base de datos en memoria sin tocar datos de producción. SQLite – Base de datos ligera perfecta para entornos de prueba aislados.	Pruebas E2E Cypress – Pruebas exhaustivas de aplicaciones web con depuración potente y recarga en tiempo real. Supertest – Pruebas de solicitudes HTTP específicamente diseñadas para la validación de REST API.
--	--	---

Comprendiendo algunos componentes clave

 Mocha Framework de pruebas que proporciona estructura con los hooks <code>describe</code>, <code>it</code>, <code>before</code> y <code>after</code> para organizar conjuntos de pruebas.	 Chai Biblioteca de aserciones que ofrece múltiples estilos (<code>expect</code>, <code>should</code>, <code>assert</code>) para una validación de pruebas expresiva.	 Supertest Biblioteca de aserciones HTTP para realizar solicitudes y probar endpoints de API con facilidad y claridad.
---	--	---

Una estrategia de pruebas bien estructurada no solo detecta errores temprano, sino que también sirve como documentación viva para tu base de código, facilitando significativamente el mantenimiento y la incorporación de nuevos miembros.